SIC 8 / 2026 · BIG DATA ENGINEERING TRACK

Graduation Project Documentation

Enterprise Fleet Telematics Lakehouse & Dynamic Logistics Warehouse



Project 03 · Supply Chain & Intelligent Logistics

Data notice

All vehicle, driver, trip, telemetry and anomaly data described in this document is simulated. Figures reported here describe this simulation and its implemented rules; they are not evidence of real-world fleet incidents or regulatory compliance.

Prepared for project review and final demonstration

SIC 8, 2026

Table of Contents

1. Executive Summary
2. Business Problem and Objectives
3. Project Scope and Domain Requirements
4. End-to-End Architecture
5. Source Data and Generation
6. Data Ingestion
7. Bronze Layer — Preserving Raw Evidence
8. Silver Layer — Validation, Enrichment and Anomaly Detection
9. Gold Layer — Dimensional Model and KPIs
10. Orchestration and Execution
11. Power BI Dashboard — Turning Data into Business Insight
12. Validation and Measured Evidence
13. Business Value and Conclusion
14. Team Responsibilities

1. Executive Summary

Fleet operators cannot manage what they cannot see. A delivery fleet produces a constant stream of signals — where a vehicle is, how fast it is going, whether its engine is healthy, and whether the cargo it carries is still at a safe temperature — but these signals arrive from separate systems and are too detailed, individually, for a manager to act on. This project builds a complete data platform that takes that raw telemetry, through a real source system and a cloud lakehouse, and turns it into a small set of trusted answers on a Power BI dashboard.

The system simulates a delivery fleet of 5,000 vehicles (about 1,500 refrigerated trucks and 3,500 dry vans) operating on 30 fixed road corridors between 10 depots. Vehicles report GPS position, speed, engine data and fuel use every 30 seconds; refrigerated trucks also report cargo temperature. The simulation deliberately injects realistic problems — cold-chain breaches, excessive idling, route deviations, engine overheating, and several categories of malformed data — so that the pipeline has something real to catch, and so the team can prove, end to end, that nothing is silently lost.

The platform follows the Medallion architecture required by the SIC 8 Big Data Engineering handbook: a Bronze layer preserves every event exactly as it arrived; a Silver layer validates, deduplicates, enriches and flags anomalies; and a Gold layer organizes the result into a star schema and a set of business KPI tables. Microsoft Fabric hosts the cloud side (Eventstream, OneLake, Delta Lakehouse, PySpark notebooks); a local Docker Compose stack — a FastAPI ingestion endpoint, a PostgreSQL source database, Debezium change-data-capture and Kafka — plays the role of the fleet company's own operational systems, matching the handbook's requirement that the source database be connected to Kafka and that data then reach the cloud.

The result is a six-page Power BI dashboard that lets a fleet manager see fuel efficiency, cold-chain compliance and anomaly activity at a glance, lets a supervisor drill into driver behaviour, and lets a data engineer or reviewer confirm — on the same dashboard — that the pipeline's quality checks passed and that the Gold numbers reconcile with Silver. This document tells that story end to end: the business problem, the architecture and the reasoning behind each technology choice, the three processing layers, the dashboard, and the evidence gathered to validate it, together with the limitations the team identified along the way.

2. Business Problem and Objectives

2.1 The business challenge

A fleet operator runs thousands of delivery vehicles on fixed routes between cities. Some trucks carry ordinary goods (dry vans); others are refrigerated and must keep their cargo cold for the whole trip. Managers need everyday questions answered: Are vehicles using fuel sensibly? Is cold cargo staying cold? Which drivers idle too long or drive too fast? Where, and when, do unusual events happen?

The SIC 8 project handbook frames this directly for the Supply Chain & Intelligent Logistics domain: vehicle diagnostics, position and cargo conditions normally arrive as separate, disconnected streams, which makes route deviations, refrigeration breaches and excessive idling difficult to investigate consistently. This project's job is to join those streams in a lakehouse and a dimensional warehouse, so that the dashboard — not a spreadsheet, and not a person manually cross-referencing three systems — is where managers see the answer.

2.2 Project objectives

- Generate repeatable, domain-specific fleet telemetry with controlled, reproducible data-quality issues, so the pipeline can be tested against known problems.
- Support both continuous (real-time) ingestion and scheduled historical/batch ingestion, because a real fleet company has both live traffic and periodic bulk loads (for example vehicles that were offline and catch up later).
- Preserve every source record and its rejection reason, so nothing is silently dropped and every quality issue is auditable and recoverable.
- Produce cleaned, deduplicated Silver telemetry and anomaly episodes, so that an "event" (for example one long idling stretch) is counted once, not once per 30-second reading.
- Publish a Gold dimensional model and KPI tables that a BI tool can query directly, with ratios stored as numerator/denominator so they are never summed or averaged incorrectly.
- Validate outputs with automated data-quality checks and Silver-to-Gold reconciliation, so the numbers on the dashboard can be trusted.
- Document measured results, assumptions and limitations honestly, rather than present a synthetic simulation as production-grade or as proof of real-world incidents.

2.3 Intended users

Audience	What they need from the system
Fleet and operations managers	Compare routes, fuel efficiency, driver behaviour, cold-chain readings, and detected anomaly episodes, in plain business terms.
Data engineers / pipeline owners	Inspect ingestion, rejected records, quality checks, reconciliation results and job durations to confirm the pipeline is healthy.
Project reviewers	Follow the pipeline end to end and judge whether outputs are reproducible and backed by evidence, per the handbook's acceptance criteria.

3. Project Scope and Domain Requirements

The SIC 8 Graduation Projects Handbook (Big Data Engineering Track) asks every team to choose one of five domains and deliver a complete, end-to-end technical system across five required pillars: data generation, ingestion, distributed processing, a lakehouse/dimensional warehouse, and orchestration with analytics. This project selected Domain 03 — Supply Chain & Intelligent Logistics — "Enterprise Fleet Telematics Lakehouse & Dynamic Logistics Warehouse."

3.1 What the domain brief requires

Requirement area	Handbook requirement	How this project addresses it
Generator & anomalies	Track 5,000 vehicles on fixed corridors; generate GPS, OBD-II fuel/RPM/engine temperature, and cargo refrigeration temperature; inject cold-chain breaches, excessive idling and route deviations.	Implemented exactly as specified, plus an additional engine-overheating anomaly the team added beyond the handbook minimum.
Processing & serving	Preserve vehicle, route and time relationships; deliver fuel efficiency per route, cold-chain compliance percentage, and driver safety/idling scorecards with documented metric definitions.	Delivered as three Gold KPI tables with explicit, documented formulas (Section 9).
Infrastructure options	Either a cloud pathway (e.g. S3/Kinesis/EMR/Snowflake) or a local pathway (Dockerized Kafka, HDFS/MinIO, local PySpark, Trino), with the shared lakehouse, quality-gate and orchestration pillars either way.	A hybrid of both: the source system (API, PostgreSQL, Kafka/CDC) runs locally in Docker Compose, matching a real company's own on-premises operational systems, while Microsoft Fabric hosts the cloud lakehouse, processing and BI layer (Section 4).

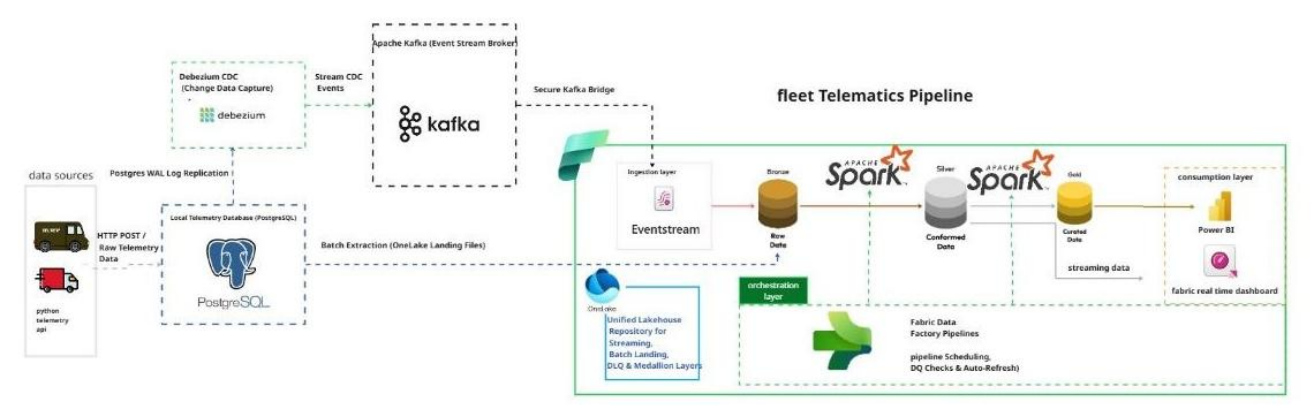
3.2 Minimum acceptance criteria

- Configuration and output evidence cover 5,000 distinct vehicles.
- All specified telemetry fields and the required anomaly families are present in the generated data.
- Route, cold-chain and driver KPIs reconcile with analytical queries.
- A complete, orchestrated run connects raw events to the warehouse and the dashboard, with evidence retained at each step.

4. End-to-End Architecture

4.1 Architecture at a glance

The implemented design is hybrid. The simulated fleet and its operational source systems — the Python generator, the FastAPI ingestion endpoint, PostgreSQL, Debezium change-data-capture and Kafka — run locally in Docker Compose, standing in for a real fleet company's own IT estate. Microsoft Fabric hosts everything downstream of that: the cloud ingestion endpoint (Eventstream), the OneLake landing area for batch files, the Bronze/Silver/Gold Delta lakehouse, and the Power BI analytics layer.



4.2 Why a hybrid design

Microsoft Fabric cannot itself host PostgreSQL, Kafka or a custom API; a fully cloud-native version of this architecture would require a separate Azure subscription (Azure Database for PostgreSQL, Event Hubs, Container Apps) running essentially the same code. Given that the handbook explicitly states infrastructure cost does not earn marks, and that running the database and broker locally is the standard enterprise pattern — operational systems on-premises, analytics platform in the cloud — the hybrid design was judged the most realistic and cost-effective way to satisfy the requirement.

4.3 Architecture components and why each was chosen

Component	Responsibility	Why it was chosen
Python generator	Produces fleet reference data (vehicles, drivers, depots, corridors, trips) and realistic telemetry, with reproducible, seeded anomalies and malformed events.	A seeded, reproducible generator gives byte-identical output for the same configuration — essential for demonstrating pipeline behaviour without relying on real vehicle data.
FastAPI ingestion API	Exposes POST /v1/telemetry?channel=live|history as a single entry point; validates every line and stores it with is_valid / reject_reason.	Represents the single entry point a real fleet company's systems would expose, instead of notebooks performing ingestion directly, per the supervisor's requirement.
PostgreSQL	Stores telemetry_live and telemetry_history, plus	Represents the operational "source system" a real company would run before any data reaches an analytics platform.

Component	Responsibility	Why it was chosen
	ingestion/extraction logs; insert-only (UPDATE/DELETE revoked).	
Debezium CDC + Kafka	Publishes every new telemetry_live row to a Kafka topic without the API needing to know about Kafka.	The standard, decoupled way to connect a relational database to a streaming broker.
Bridge service	Consumes Kafka and forwards events to Fabric Eventstream; commits Kafka offsets only after the cloud acknowledges receipt; retries with 1-2-4-8s backoff, then a dead-letter file.	Guarantees at-least-once delivery to the cloud and survives restarts without losing or duplicating data (demonstrated in Section 12).
Hourly extractor	Exports telemetry_history in one-hour slices to OneLake as gzip JSON Lines, tracked in an extract log.	Matches the handbook's "scheduled extractor scripts for periodic landing" requirement and makes incremental loads, re-runs, crash/retry and backfills natural operations rather than special cases.
Microsoft Fabric Eventstream	Cloud-managed entry point for the live Kafka stream; writes into the Bronze Delta table.	A managed cloud ingestion service, avoiding the team having to operate cloud-side broker infrastructure.
OneLake	Shared storage for historical landing files and all lakehouse Delta tables.	Common storage layer for both the batch and streaming paths into Bronze.
Delta Lake / Fabric Lakehouse	Stores Bronze, Silver and Gold tables with ACID transactions, schema enforcement and time travel.	Required by the handbook's Lakehouse pillar; supports MERGE-based idempotent loads and historical (time-travel) queries.
PySpark / Spark SQL	Parses, validates, deduplicates, enriches, detects anomalies, aggregates episodes, and builds the Gold model.	The handbook's required distributed-processing engine; makes every transformation explicit and auditable, and scales to millions of events.
Power BI	Presents KPIs, route/driver analysis, anomaly investigation and pipeline health in a six-page report.	Makes curated Gold outputs accessible to operational managers and reviewers without requiring SQL knowledge.

4.4 The Medallion pattern

The project follows the handbook's required Medallion architecture throughout: Bronze preserves ingested records exactly as received (including malformed ones, kept visible rather than dropped); Silver produces curated, deduplicated telemetry together with anomaly detection outputs; and Gold serves business-oriented, query-ready tables to Power BI.

5. Source Data and Generation

Everything downstream depends on a realistic, reproducible fleet simulation. This section describes what the Python generator produces and why its design choices matter for later KPIs.

5.1 The simulated fleet

Item	Value
Vehicles	5,000 (1,500 refrigerated trucks, 3,500 dry vans), IDs VEH-00001 … VEH-05000
Drivers	5,200, assigned one per vehicle
Depots	10, across Cairo, Alexandria, Port Said, Suez, Ismailia, Tanta, Mansoura, Minya, Asyut and Hurghada
Corridors (fixed routes)	30, between depot pairs, each with a planned distance and a reefer temperature range (2–8 °C)
Trips (reference)	10,000 planned trips
Telemetry interval	Every 30 seconds while a vehicle is on a trip
Units / format	Metric only (km, km/h, rpm, °C, L/h); UTC ISO 8601 timestamps; WGS84 decimal-degree coordinates; JSON Lines
Reproducibility	Fixed seed (42); the pilot generator's documentation states identical seed/configuration runs produce byte-identical output
Personal data	None — driver names are random combinations of common first/last names

5.2 Telemetry event schema

Each reading is a single JSON Lines event with the following documented fields:

Field	Meaning	Notes
event_id	Unique reading identifier	UUID string
vehicle_id	Vehicle identifier	VEH-00001 … VEH-05000
trip_id	Associated trip	References trip reference data
event_time	Time the reading was taken	ISO 8601 UTC string
latitude, longitude	GPS position	Decimal degrees
speed_kmh	Vehicle speed	km/h
rpm	Engine speed	0 represents engine off
engine_temp_c	Engine temperature	°C
fuel_rate_lph	Fuel consumption rate	litres/hour
odometer_km	Cumulative odometer	km; expected not to decrease

Field	Meaning	Notes
cargo_temp_c	Cargo temperature	°C; null is normal for dry vans

5.3 Injected anomalies and deliberate data-quality issues

To prove the pipeline actually catches problems rather than simply passing clean data through, the generator deliberately injects both operational anomalies and malformed records, each recorded in an evaluation "truth" file that detection code is never allowed to read.

Case	How it is simulated	Intended handling downstream
Cold-chain breach	Refrigerated cargo temperature pushed above the 2–8 °C range for several consecutive readings	Detected as a sustained out-of-range episode
Excessive idling	Vehicle holds position, speed ≈ 0, engine running (rpm > 0)	Detected as a sustained idle period (not a planned, engine-off stop)
Route deviation	GPS position pushed sideways off the planned corridor	Flagged when position stays off-corridor for several consecutive readings
Engine overheating	Engine temperature driven into an abnormal range	Detected with a rule-based threshold (added by the team, beyond the handbook minimum)
Malformed JSON	Truncated JSON text	Preserved as raw text in Bronze; rejected with a reason in Silver
Missing / null required field	A required field is omitted or set to null	Rejected from curated telemetry; reason retained
Wrong type	e.g. "speed_kmh":"fast"	Rejected during schema/type validation
Impossible sensor value	e.g. engine_temp_c: 900	Kept visible for audit rather than silently deleted (Section 9.5)
Duplicate	Repeated event	Removed using the configured deduplication key (db_id and/or vehicle_id + event_time)
Late event	event_time earlier than its arrival time	Retained and ordered analytically by event_time

5.4 From pilot to v2

The project was built in two generations. The pilot proved the generator and a direct Eventstream ingestion path (notebooks → bronze_telemetry) on one simulated day. Following supervisor feedback, v2 added the API/PostgreSQL/Kafka source system described above, and extended the simulated window to four days of trips (40,000 trips total) split across a batch-loading day, a batch-incremental-and-failure day, and a dedicated streaming "golden run" day — explicitly to reach the data-volume levels required by the handbook's testing guide. The pilot's data and Bronze table (bronze_telemetry) were kept unchanged and separate from the v2 table (bronze_telemetry_v2); they should never be described as one table.

6. Data Ingestion

The handbook's Data Ingestion pillar asks for a high-velocity broker (or managed equivalent), scheduled extraction for periodic landing, an immutable raw zone, and demonstrable dead-letter handling, ingestion logs and retry backoff. This project implements all of these twice over: a direct streaming path into the cloud, and a batch/history path through the local source system — reflecting how a real fleet's live telemetry and its catch-up/backfill data typically travel differently.

6.1 The v2 source system: API, PostgreSQL, Kafka

The ingestion API exposes a single endpoint, POST /v1/telemetry?channel=live|history, accepting one raw JSON event per line. Every line is validated using the same rules as the pilot's own validation step, and every line — valid or not — is stored, tagged with is_valid and a reject_reason, inside one transaction per request, with a row logged in api_ingest_log. This matters because it means nothing submitted to the API is silently discarded: bad data stays visible with the reason it failed.

PostgreSQL holds two tables sharing one ID sequence (so every row has a globally unique db_id): telemetry_live for the real-time feed, and telemetry_history for bulk/backfill uploads. Both tables are insert-only — UPDATE and DELETE privileges are revoked — so PostgreSQL itself behaves like the "source system" the supervisor asked for, not a mutable staging area.

Debezium change-data-capture watches telemetry_live and turns every new row into a Kafka message (topic fleet.public.telemetry_live, 4 partitions, KRaft mode) without the ingestion API needing any awareness of Kafka. This is the standard way to connect an operational database to a streaming platform, and it means the API's only job is to write to Postgres correctly.

6.2 Getting the stream to the cloud: the bridge

A dedicated bridge service consumes the Kafka topic and forwards events to Microsoft Fabric Eventstream over its Kafka-compatible endpoint (SASL_SSL). Critically, the bridge commits its Kafka consumer offsets only after the cloud has acknowledged receipt, and retries failed sends with 1-2-4-8 second backoff before writing to a local dead-letter file. This single design decision is what makes the bridge safe to restart mid-run without losing or duplicating data — demonstrated directly in the streaming golden run (Section 12.2).

6.3 The batch/history path: hourly extractor

History data (for example, a vehicle that was offline and later catches up) is written to telemetry_history through the same API. An hourly extractor then exports one hour slice at a time to Files/landing_v2/event_date=…/event_hour=… as gzip JSON Lines, tracking every hour's status (started → done) in extract_log. Processing one hour at a time, rather than the whole table, is what makes incremental loads, re-runs, crash recovery and historical backfills all the same operation instead of special cases — each is just "process this hour again." A notebook loader then inserts each extract file's rows into bronze_telemetry_v2 exactly once per db_id and archives the file, so accidental re-runs or backfills of an already-loaded hour never create duplicates (proven in Section 12.1, Run 5).

6.4 Dead-letter handling, logging, and retries

Both the pilot and v2 pathways keep rejected or failed events visible rather than discarding them:

- Pilot: a dlq_telemetry table separates sender-level failures (network/Eventstream unavailable, caught after 1-2-4-8s retries) from validation-level failures (malformed JSON, missing/null fields, wrong types), each with an explicit reason; a bronze_ingestion_log table records every sender run, 10-second interval, and validation pass.
- v2: the API records is_valid and reject_reason on every row inline; the bridge keeps a dead-letter file for sends the cloud never acknowledged; extract_log and landing_files_loaded give an auditable, idempotent record of every batch load.

6.5 Orchestration trigger

Every batch stage (extraction, then the Fabric loader notebook) is started through platform/fabric/run_notebook.py, a thin wrapper around the Fabric REST API that launches a notebook job and waits for it to finish. This is the mechanism referenced throughout Section 10.

7. Bronze Layer — Preserving Raw Evidence

Bronze is the system's "nothing is lost" guarantee. Two Bronze tables exist, reflecting the two generations of the project, and they are never merged or described as one table:

Table	Populated by	Immutability
bronze_telemetry (pilot)	Fabric Eventstream (stream path) and the Files/landing batch path	Append-only Delta configuration (delta.appendOnly = true); proven directly from the Delta transaction log, not just DESCRIBE HISTORY, because the Eventstream writer leaves the operation field empty
bronze_telemetry_v2	Fabric Eventstream for the Kafka/CDC live path; notebook 11_load_landing_v2 for history extracts	Insert-only loader logic (one insert per db_id, files archived after load); the source PostgreSQL tables also have UPDATE/DELETE revoked

Every Bronze row carries both the event and its ingestion metadata — raw payload, source (kafka_cdc vs db_extract, or stream vs batch in the pilot), sent_at_utc, run_id, db_id, is_valid, reject_reason, arrival time, and (for the Kafka path) partition/offset — so Silver can tell, for any row, exactly how and when it arrived. The deduplication key used downstream, db_id, exists specifically because v2's Kafka delivery is at-least-once by design: the same event can legitimately arrive at Bronze more than once (for example after the bridge restarts), and db_id is what lets Silver and Gold collapse that back down to one logical reading.

7.1 Append-only proof (pilot)

Because the Fabric Eventstream writer does not record an operation name, DESCRIBE HISTORY alone cannot prove nothing was deleted or overwritten. The pilot's verification notebook instead reads the Delta transaction log directly (every _delta_log/*.json commit, which always records files added or removed, regardless of who wrote them) and checks two things: that no commit ever removed data, and that every file ever added to the table is still part of it. Both checks passed on the final run; the same checks were also tested against a table where rows had deliberately been deleted, and both correctly failed there — confirming the proof can actually detect a violation, not just pass by default. The table was then locked with delta.appendOnly = true, and DELETE/UPDATE statements were confirmed to be refused by Delta afterward.

8. Silver Layer — Validation, Enrichment and Anomaly Detection

Silver is where raw Bronze events become trustworthy, analysis-ready telemetry. The implementation is a Spark Structured Streaming job reading from bronze_telemetry_v2, processing each micro-batch through parsing, validation, deduplication, incremental write, enrichment, movement analytics and anomaly detection.

8.1 Schema parsing

An explicit PySpark schema (StructType) is defined for the telemetry JSON payload — event_id, vehicle_id, trip_id, event_time, latitude, longitude, speed_kmh, rpm, engine_temp_c, fuel_rate_lph, odometer_km, cargo_temp_c — so Spark does not have to infer structure on every micro-batch. from_json parses the raw column against this schema, the parsed fields are selected alongside Bronze's own metadata (db_id, source, is_valid, reject_reason, arrival time), and event_time is cast from string to a proper timestamp, which every later stage (ordering, partitioning, episode grouping) depends on.

8.2 Data validation and rejected records

A record must have non-null event_id, vehicle_id, event_time, latitude, longitude and speed_kmh to be considered structurally valid — these are the fields needed to identify an event, attach it to a vehicle, order it in time, and locate and evaluate its movement. A record is rejected into silver_rejected if Bronze's own is_valid flag is false, or if it fails this condition; the rejection reason is taken from Bronze's reject_reason where available, with a fallback description otherwise. Keeping rejected records in their own table (rather than discarding them) lets data-quality problems be quantified and investigated without contaminating the curated dataset.

8.3 Deduplication

Two different deduplication rules are applied, because duplicates can arise for two different reasons:

Key	Purpose	Scope
db_id	Identifies a source database row for ingestion-level deduplication (handles Kafka's at-least-once delivery)	Within each streaming micro-batch (dropDuplicates), plus a left-anti join against db_id values already present in silver_telemetry for incremental runs
vehicle_id + event_time	Identifies a telemetry observation according to the project's business definition of "one reading"	Within the valid records of the current micro-batch

8.4 Incremental processing

Rather than re-processing the whole Bronze history on every run, each micro-batch's valid, cleaned records are compared against the db_id values already present in silver_telemetry using a left-anti join, so only genuinely new records are appended. The job uses Spark's availableNow trigger, so it processes everything currently available and then terminates, rather than running as an always-on streaming service — appropriate for scheduled, notebook-driven execution.

8.5 Partitioning

silver_telemetry is partitioned by event_date (derived from event_time). This supports partition pruning for date-filtered queries and keeps date-oriented analytical workloads and incremental writes manageable — the actual performance benefit depends on data volume and query patterns, which were not separately benchmarked here.

8.6 Enrichment

Telemetry is joined to vehicle reference data (vehicle_type, make, model, home depot, assigned driver, etc.) using a broadcast join. The reference table is small enough to be distributed to every executor, avoiding a large shuffle of the full telemetry dataset; a left join is used so that telemetry is never discarded purely because a reference match is missing.

8.7 Vehicle movement analytics

A window function, partitioned by vehicle and ordered by event time, retrieves each reading's previous latitude, longitude and timestamp (lag). From these, the pipeline calculates:

- time_gap_sec — seconds since the vehicle's previous reading.
- distance_km — great-circle (Haversine) distance from the previous GPS position, valid only when the time gap is between 3 and 3,600 seconds and the distance is at least 0.01 km.
- calculated_speed_kmh — distance divided by time gap, converted to km/h; values above 180 km/h are reset to zero as an implementation-specific filtering rule (flagged as a limitation in Section 13, because it can hide the original speed from the downstream GPS-anomaly rule that checks for exactly this kind of spike).

8.8 Idling, cold-chain and anomaly detection rules

Several independent, rule-based indicators are computed per reading, then combined into a single anomaly type per record using a fixed priority order (cold-chain breach, then excessive idling, then route deviation, then engine overheating, then RPM spike) when more than one rule fires on the same reading:

Rule	Condition (as implemented in Person 2's Silver notebook)
Idling	speed_kmh < 2 and rpm > 0 (a planned stop with the engine off is not idling)
Cold-chain violation	cargo_temp_c < -2 °C or > 12 °C (the per-reading Silver rule; the Gold KPI's compliance range is the tighter, corridor-defined 2–8 °C — see Section 9)
Engine overheating	engine_temp_c > 105 °C
GPS teleportation / route-deviation signal	calculated_speed_kmh > 100 km/h and distance_km > 0.5 km
RPM spike	rpm > 4500

8.9 Anomaly episode aggregation

A single anomalous reading is not a business incident on its own — a 20-minute idling stop produces about 40 flagged readings at the 30-second interval. To avoid counting the same event many times, anomaly readings are grouped into episodes: records are windowed by vehicle and anomaly type, ordered by time, and a new episode starts whenever there is no previous reading or the gap since the last one exceeds 15 minutes. Each episode is then summarized with its start time, end time, reading count, maximum calculated speed, and duration. silver_anomaly_episodes holds 9,724 such episodes.

8.10 Anomaly detection evaluation

Detection quality is measured against a ground-truth file the detection logic never reads. The result recorded in Person 2's documentation:

Metric	Result
True positives	5
Total detected in scope	5
Total ground-truth episodes	6
Precision	1.0000
Recall	0.8333
F1-score	0.9091

9. Gold Layer — Dimensional Model and KPIs

Gold is where Silver's cleaned, event-level data becomes a business-oriented star schema that Power BI can query directly, with ratio metrics pre-split into numerator and denominator so they are always correct however a user filters the report. The layer is built by four Spark notebooks run in order — NB01 (dimensions), NB02 (facts), NB03 (quality checks), NB04 (KPIs and reconciliation) — all using idempotent Delta MERGE writes, so re-running a notebook never creates duplicates.

9.1 Star schema overview

Six dimension tables describe the business entities (date, time, vehicle, driver, route, anomaly type); three fact tables record events at three different grains (one 30-second reading, one trip, one anomaly episode); three KPI tables hold daily, pre-aggregated business numbers; and three operational tables record pipeline health.

Table	Grain (what one row represents)	Role
gold_dim_date	One calendar day	Date filtering and trend analysis
gold_dim_time	One minute of the day (1,440 rows)	Time-of-day and day-part analysis
gold_dim_vehicle	One vehicle	Vehicle type and context (Type 1 — overwrite)
gold_dim_driver	One driver version	Driver name, depot, risk level, and history (Type 2 — see 9.3)
gold_dim_route	One corridor/route	Route names and comparisons (Type 1)
gold_dim_anomaly_type	One anomaly type, plus a "none" placeholder	Anomaly labels and classification
gold_fact_telemetry	One 30-second reading, partitioned by date	Reading-level measures (not currently visible in the supplied Power BI model view — see Section 13)
gold_fact_trip	One trip	Trip duration, idle minutes/share, refrigerated readings
gold_fact_anomaly_episode	One detected anomaly episode	Episode count, duration, time of day, investigation detail
gold_kpi_fuel_efficiency_route	Route × day	Distance, fuel, fuel-efficiency numerator/denominator
gold_kpi_cold_chain	Route × day	In-range readings, total readings, compliance, breach episodes
gold_kpi_driver_scorecard	Driver version × day	Idling, excessive-idling episodes, speeding readings, route deviations
gold_quality_results	One quality check per run	Pipeline health (Section 9.5)
gold_reconciliation_results	One KPI metric per run	Silver-to-Gold reconciliation (Section 9.6)

Table	Grain (what one row represents)	Role
gold_job_run_times	One job run	Job execution durations

Note: The three operational tables (gold_quality_results, gold_reconciliation_results, gold_job_run_times) are standalone, log-style tables and are deliberately not related to the rest of the model — they describe the pipeline's own health, not the fleet's.

9.2 Why ratios are stored as numerator and denominator

Metrics such as kilometres-per-litre or cold-chain compliance percentage cannot be correctly summed or averaged across rows — averaging a set of per-route percentages does not equal the true fleet-wide percentage. Gold therefore stores the "top" and "bottom" of every such ratio (e.g. total_km and total_litres) and the division happens only in the Power BI measure, on whatever the report is currently filtered to. This is why the dashboard's totals stay correct for any selection of routes, days or drivers.

9.3 Slowly changing dimension: driver history (Type 2)

gold_dim_driver tracks changes to a driver's home depot, risk level and license class using Type 2 history — rather than overwriting a driver's record, a changed driver gets a new row, and the old row is closed off with an effective_to timestamp. The first load (ref_drivers_v1) sets every driver's effective_from to 2026-01-01; the second load (ref_drivers_v2) represents 416 drivers whose attributes changed, with the change applied at the single load-time timestamp 2026-11-03 12:00 UTC (the source file itself does not carry a per-row change date, so this cutover time is supplied by the Gold pipeline, not the raw data). Facts join to the driver version that was in effect at the time of the reading, so a scorecard split by depot or risk level reflects the correct version on each side of that cutover.

9.4 KPI definitions

KPI	Formula	Notes
Fuel efficiency (km/L)	SUM(distance_km) / SUM(fuel_used_l), by route and day	Intervals longer than 300 seconds and GPS jumps (speed > 200 km/h) contribute zero distance and fuel, so one bad reading cannot distort a route's figure.
Cold-chain compliance (%)	SUM(readings_in_range) × 100 / SUM(readings_total)	Counts only refrigerated-truck readings between 2–8 °C inclusive; dry vans are excluded entirely (not counted as 0% or 100%). A route with no refrigerated readings has a NULL compliance value.
Driver safety / idling scorecard	Four additive measures per driver version per day: idling_minutes, excessive_idling_episodes (idling ≥ 10 minutes), speeding_readings (speed > 90 km/h), route_deviations (> 2 km off-corridor for 3+ readings)	Rows are keyed by driver_key (one per driver version), so a per-person total requires grouping by driver_id.

9.5 Data quality gate

NB03 runs a battery of automated checks against every Gold fact table before KPIs are allowed to be built, and appends every result to gold_quality_results with a run_id: that every foreign key resolves in its dimension, that no fact has null keys, that no fact has duplicate business-key rows, that measures fall within valid ranges (no negative distance/fuel/idle time, interval ≤ 300 s, in-range counts ≤ total counts), that exactly one current row exists per driver, and that fact row counts match the equivalent Silver counts. A FAIL on any blocking check raises an exception and stops the pipeline — demonstrated deliberately with a "demo failure" run that feeds corrupted data (an orphan foreign key, a null date key, a negative speed, duplicate rows) through the same checks to prove they actually catch problems; these demo rows are stored separately and never block real KPI builds.

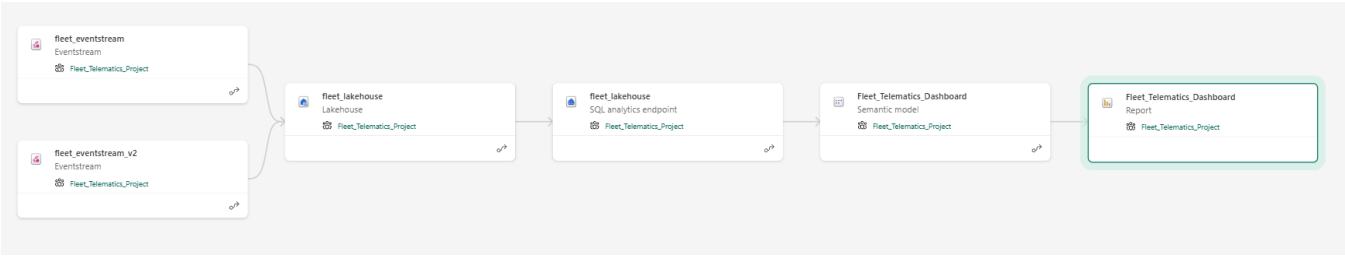
One check, range_sensor_values (plausible ranges for speed, RPM, engine temperature, fuel rate, GPS coordinates and cargo temperature), is deliberately configured as a warning rather than a blocking failure, because the project's design intentionally keeps out-of-range sensor values visible for audit rather than silently deleting them.

9.6 KPI reconciliation

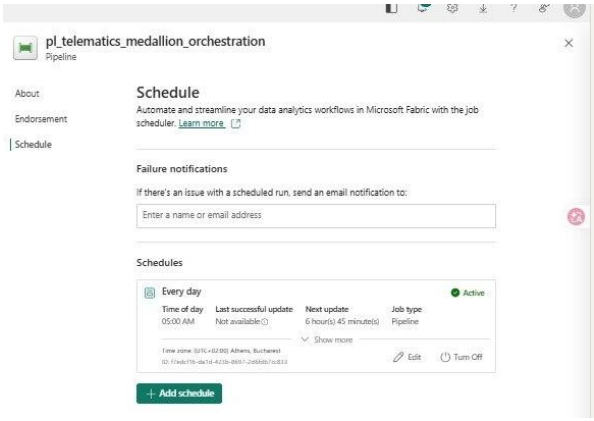
NB04 independently recomputes every KPI directly from Silver (silver_telemetry_anomalies and silver_anomaly_episodes) using the same shared business-rule expressions as the Gold build, and compares the two results at a relative tolerance of 1e-6. Mismatches would indicate a bug in the Gold transformation logic rather than a data problem. NB04 also demonstrates Delta Lake time travel (DESCRIBE HISTORY, VERSION AS OF) on the driver dimension, the trip fact and the driver scorecard KPI table, directly showing the lakehouse's ACID and historical-query capabilities required by the handbook.

10. Orchestration and Execution

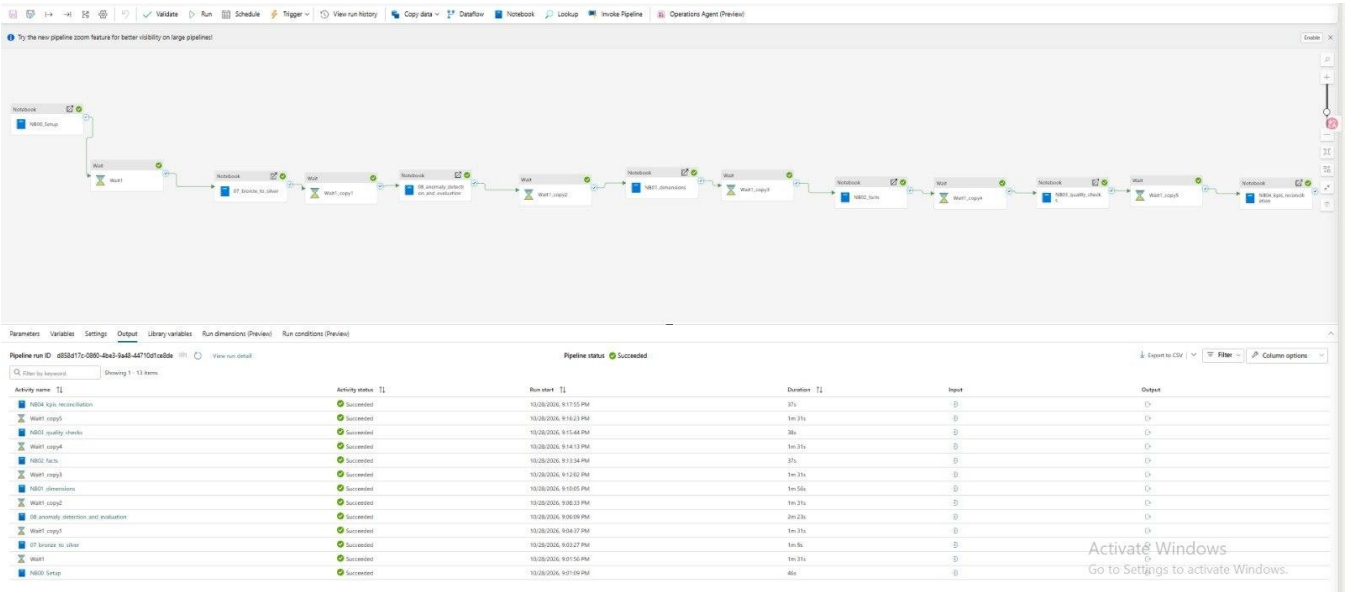
The Fabric pipeline, pl_telematics_medallion_orchestration, automates the execution of the Bronze-to-Silver-to-Gold processing workflow, including anomaly detection, dimension and fact table creation, data-quality checks, and KPI reconciliation. The pipeline is scheduled to run daily at **05:00 AM Cairo time**.



[Fabric workspace lineage showing Eventstreams, Lakehouse, SQL analytics endpoint, semantic model, and Power BI report]

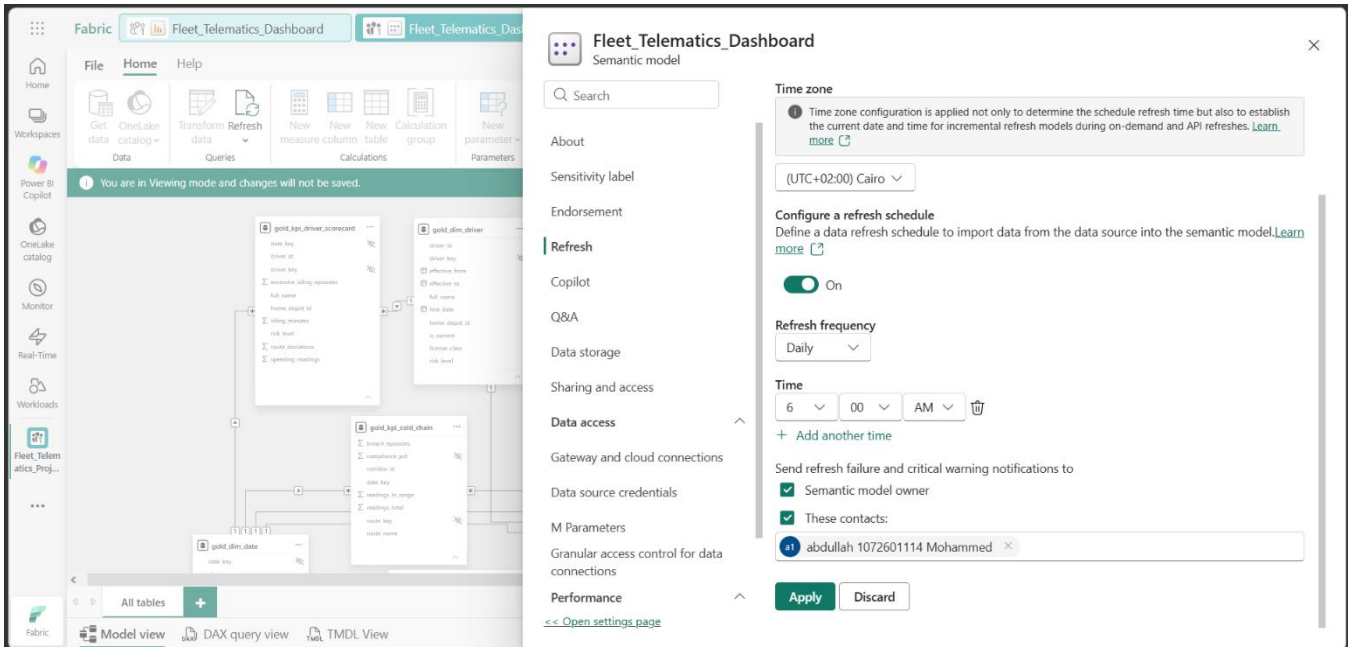


[Complete pipeline orchestration showing notebook activities and execution dependencies]



Successful pipeline run showing completed activities and overall status

The Fleet_Telematics_Dashboard semantic model uses Import mode and is configured for a separate daily refresh at **06:00 AM Cairo time**, allowing time for the pipeline to process the data before the dashboard refresh begins.



[Semantic model refresh schedule showing Daily frequency, 06:00 AM, and Cairo time zone]

This configuration automates data processing and dashboard refresh. Since the two schedules operate independently, pipeline and semantic model refresh histories should be checked to confirm that the latest processed data is reflected in the report.

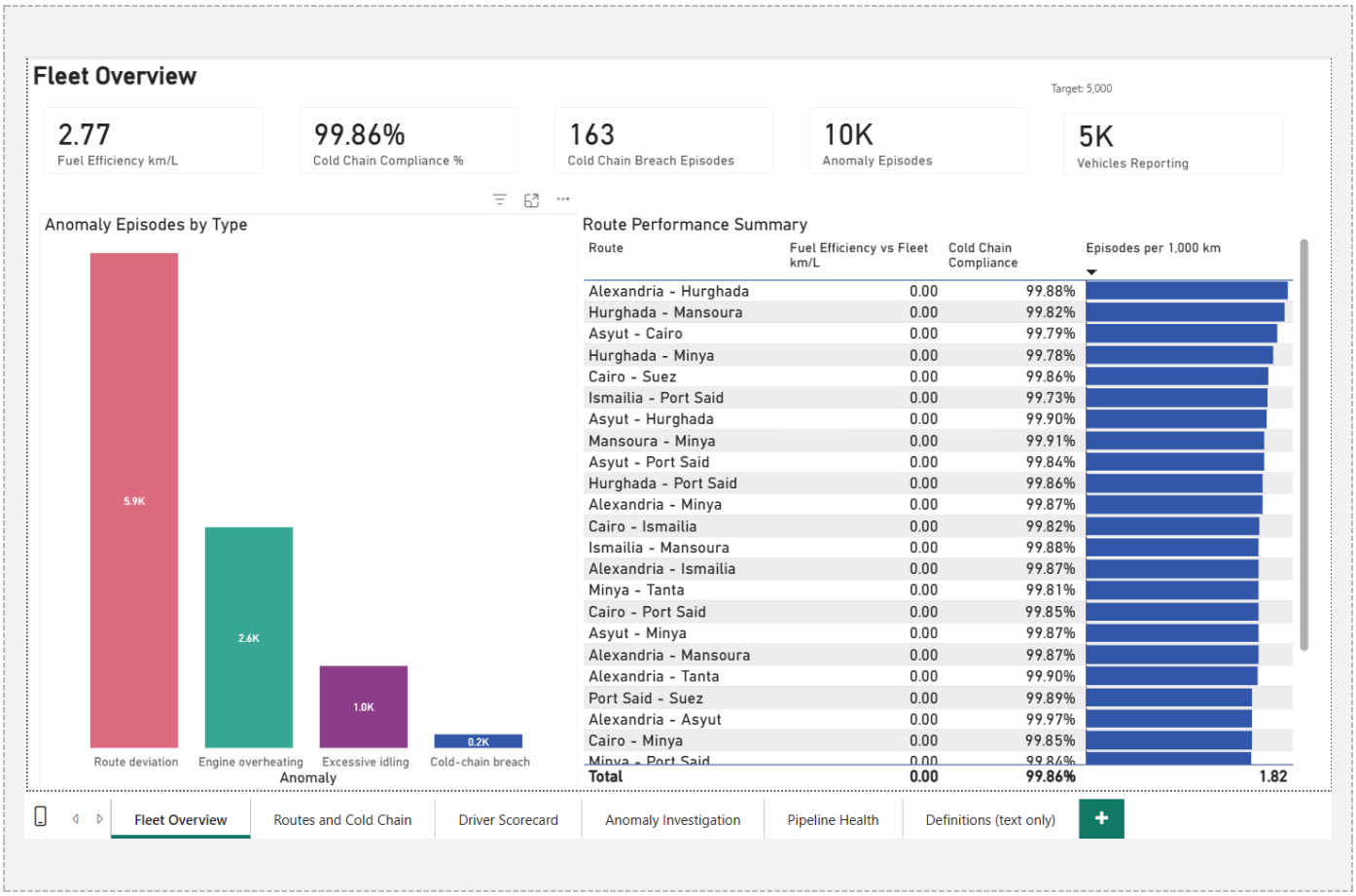
11. Power BI Dashboard — Turning Data into Business Insight

Raw telemetry is millions of rows — far too much for a manager to inspect directly. The Fleet Analytics Dashboard turns the curated Gold tables into a small set of trusted answers, and — on its own pipeline-health page — shows whether those answers can be trusted. It serves two audiences at once: fleet and operations managers who need performance, safety and cold-chain answers, and data engineers or reviewers who need proof that the pipeline ran correctly and that Gold agrees with Silver. Power BI reads only the Gold layer; it never queries Bronze or Silver directly.

Area	Business question it answers	Page(s)	Headline figure
Fleet performance	How far do vehicles travel per litre of fuel, and which routes stand out?	1, 2	2.77 km/L
Driver behaviour	Which drivers idle for long periods, speed, or leave their route?	3	8.63% speeding
Cold-chain compliance	What share of refrigerated readings stayed between 2–8 °C, and where did breaches happen?	1, 2	99.86%
Anomalies	What was detected, how long did it last, and when?	1, 4	9,724 episodes
Pipeline reliability	Did quality checks pass, and do Gold and Silver agree?	5	28 PASS, 1 WARN

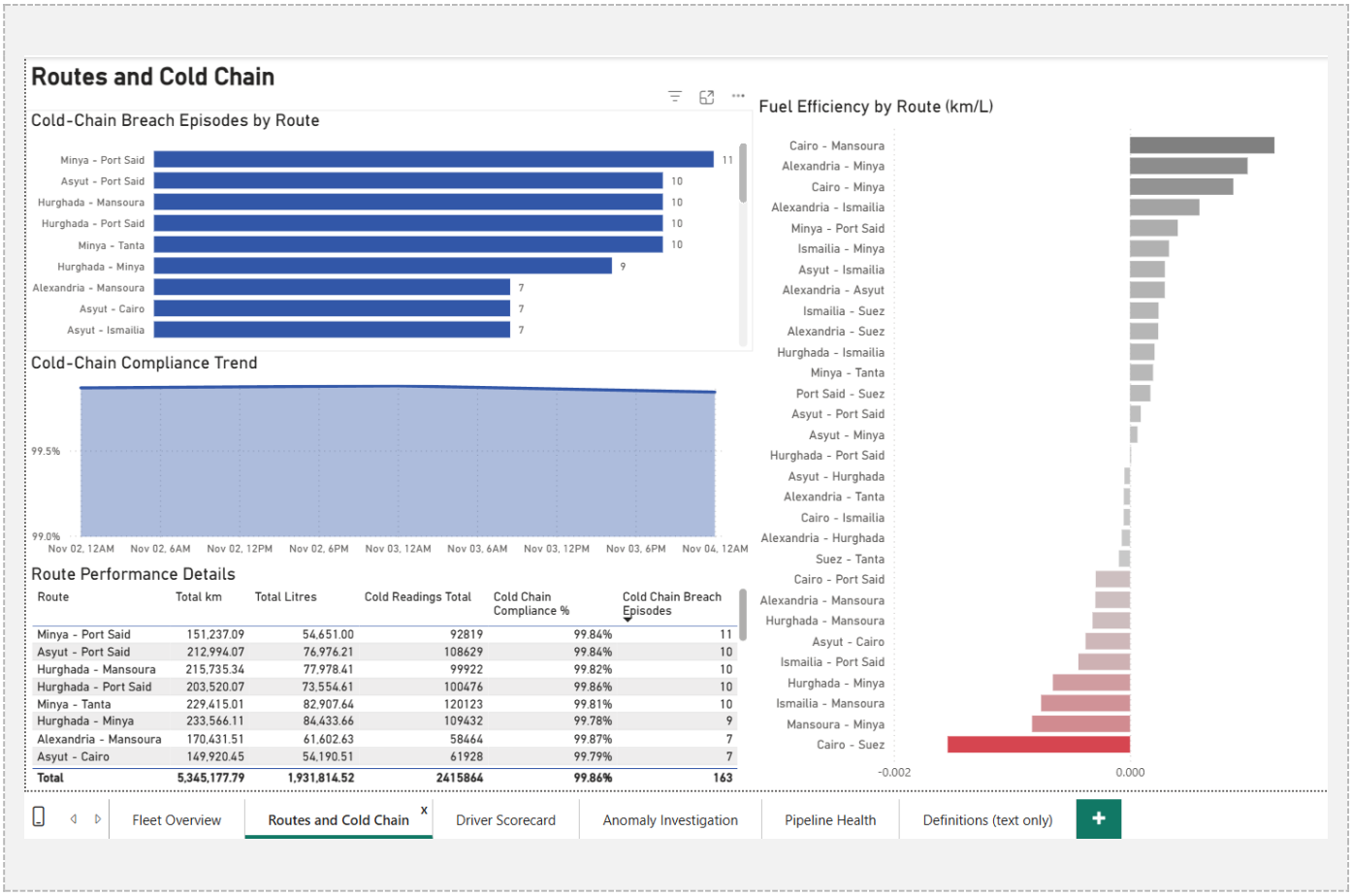
11.1 Page 1 — Fleet Overview

Purpose: give a manager the whole picture in about ten seconds. Five headline cards (fuel efficiency 2.77 km/L; cold-chain compliance 99.86%; cold-chain breach episodes 163; anomaly episodes 9,724; vehicles reporting, 5,000 target) sit above a column chart of anomaly episodes by type (route deviation 5.9K, engine overheating 2.6K, excessive idling 1.0K, cold-chain breach 0.2K) and a route performance summary table (fuel efficiency vs. fleet, cold-chain compliance, episodes per 1,000 km).



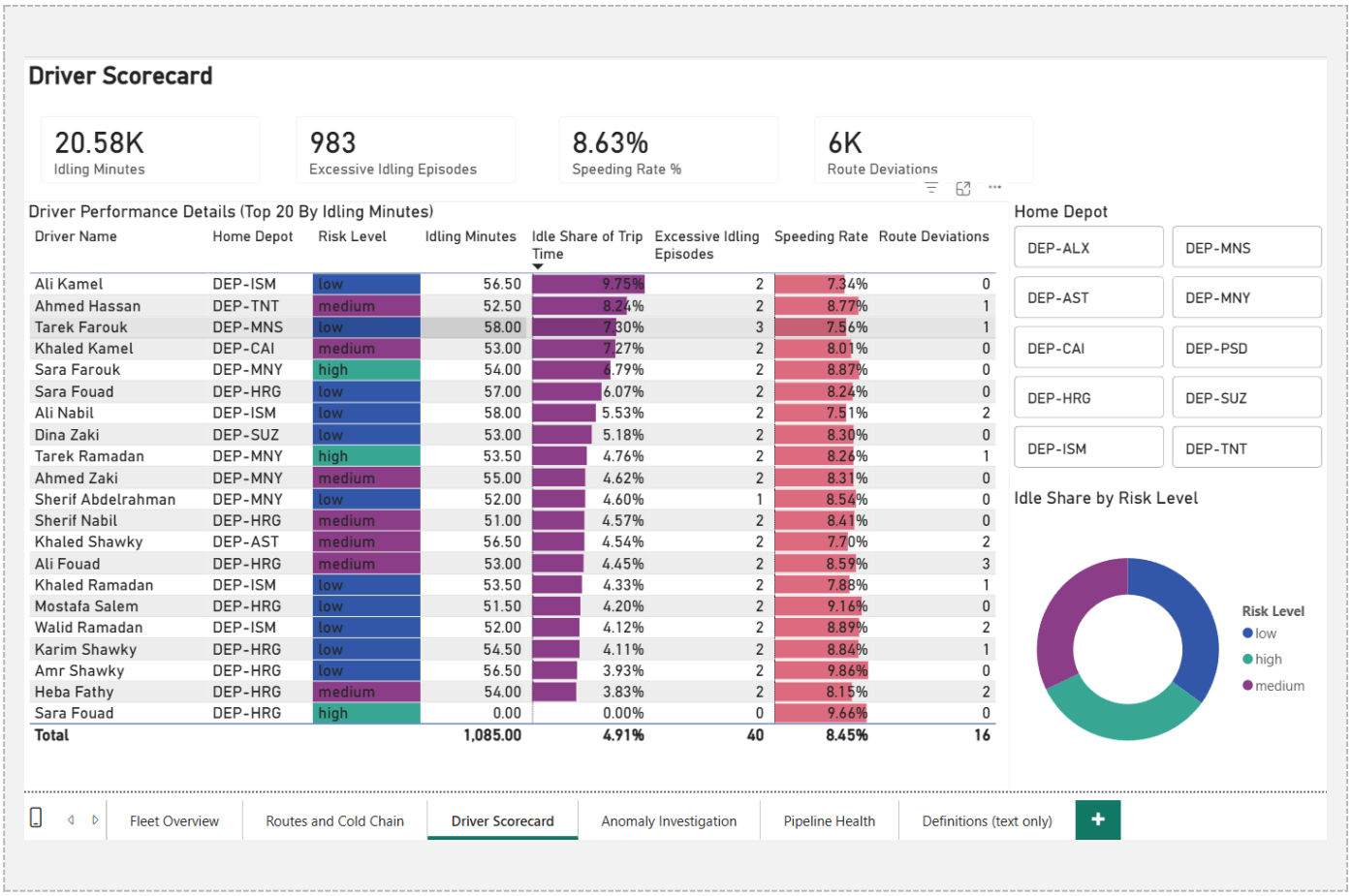
11.2 Page 2 — Routes and Cold Chain

Purpose: show which routes have the most cold-chain trouble and how each route's fuel efficiency compares with the fleet average. Visuals include a breach-episodes-by-route bar chart (Minya–Port Said leads with 11), a compliance trend by day, a diverging bar chart of each route's fuel-efficiency difference from the fleet, and a route performance details table (totals: 5,345,177.79 km; 1,931,814.52 L; 2,415,864 readings; 99.86% compliance; 163 breach episodes).



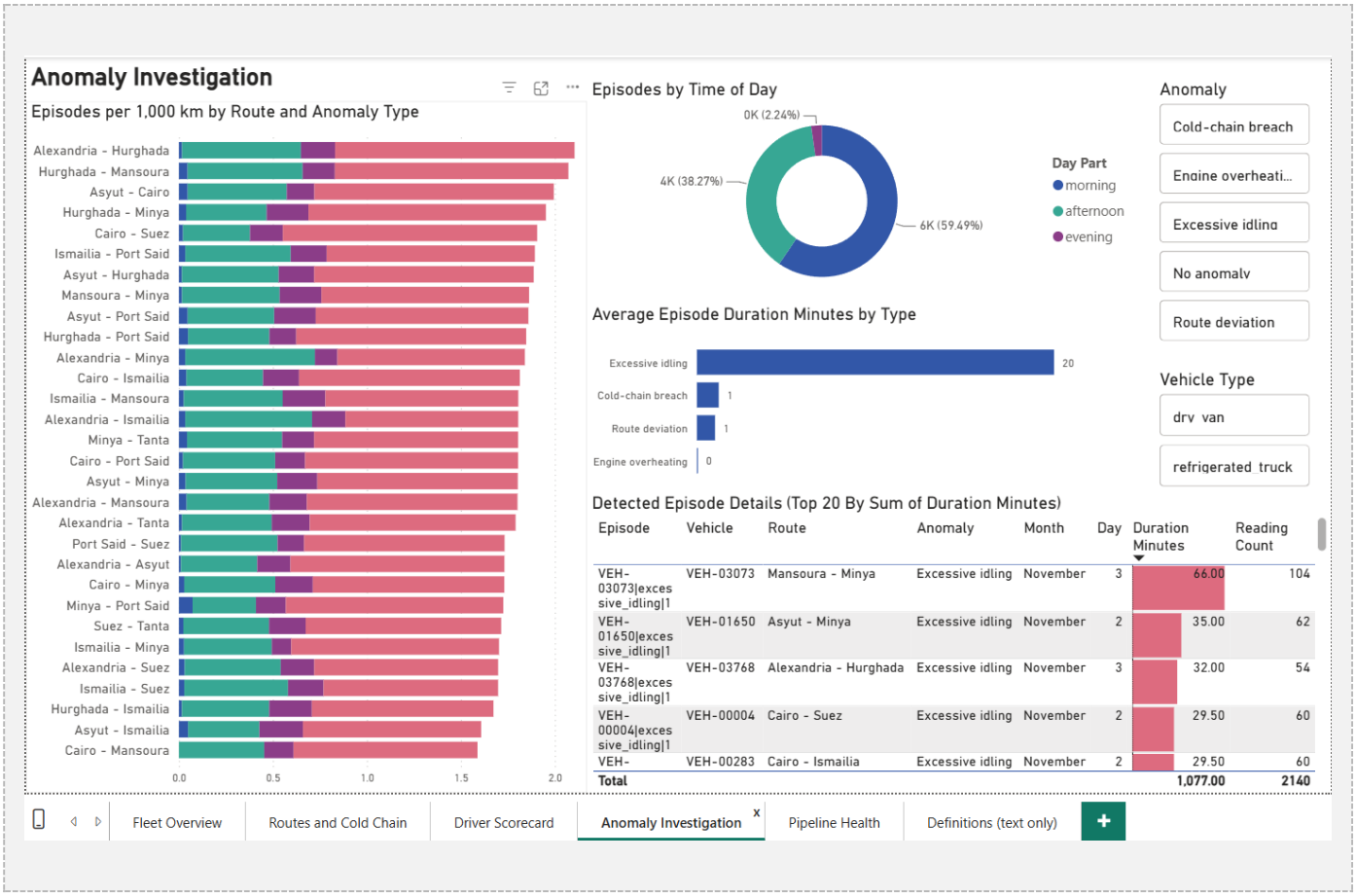
11.3 Page 3 — Driver Scorecard

Purpose: show which drivers idle most, speed most often, or leave their corridor, so supervisors can focus coaching. Headline cards show 20.58K total idling minutes, 983 excessive-idling episodes, 8.63% speeding rate and 5,932 route deviations; a Top-20 driver table and a Home Depot slicer let a supervisor drill into one location, and a donut chart splits idle share by risk level.



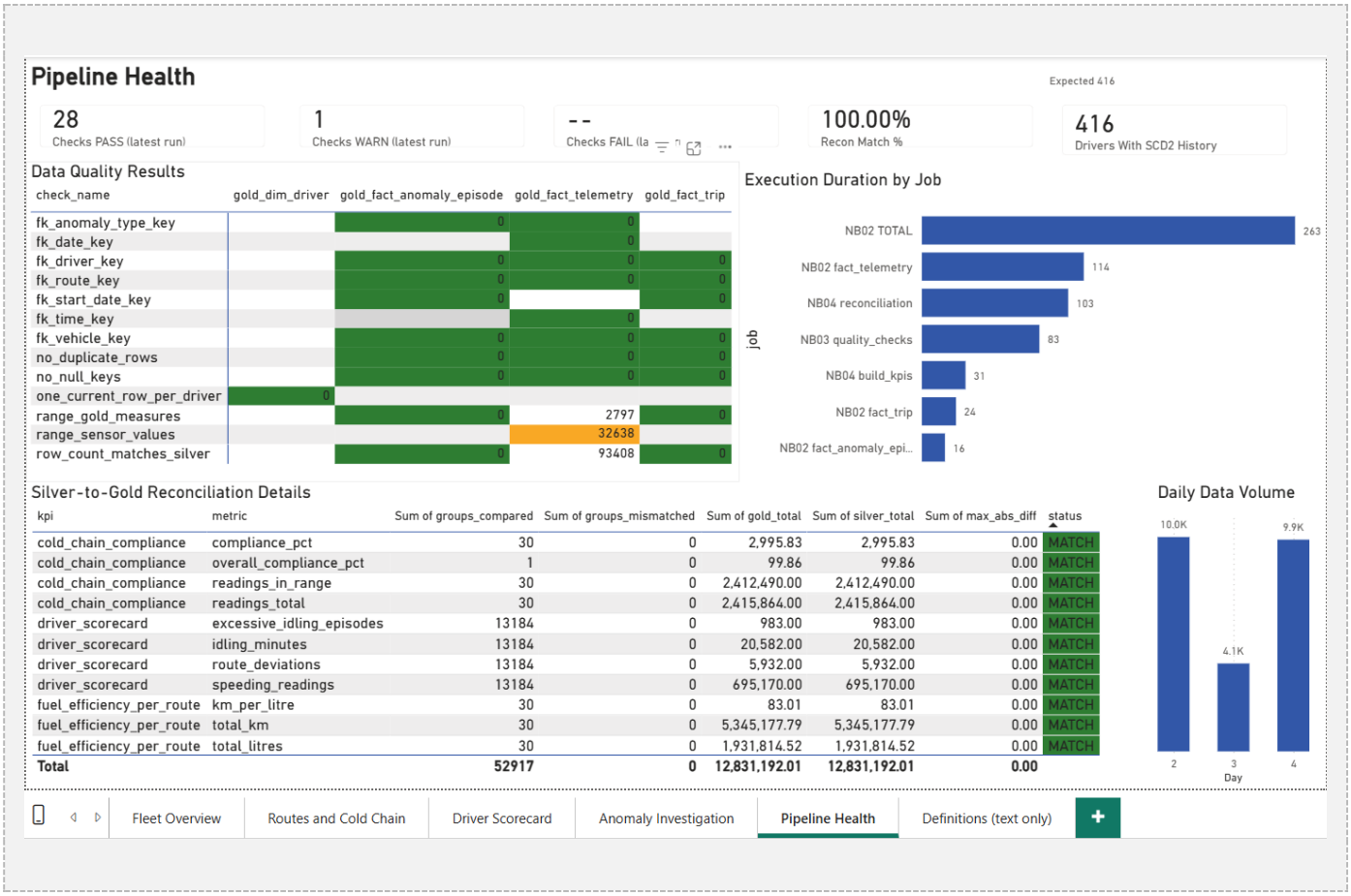
11.4 Page 4 — Anomaly Investigation

Purpose: move from "how many" to "where, when and how long," and let an analyst drill into individual episodes. A stacked bar chart shows anomaly intensity per route split by type; a donut chart shows the time-of-day split (morning 59.49%, afternoon 38.27%, evening 2.24%); a bar chart shows average episode duration by type (excessive idling ≈ 20 minutes; the others close to 1 minute, since those rules only require 3 consecutive readings); and a details table lists the longest individual episodes (the longest shown: 66 minutes, excessive idling).



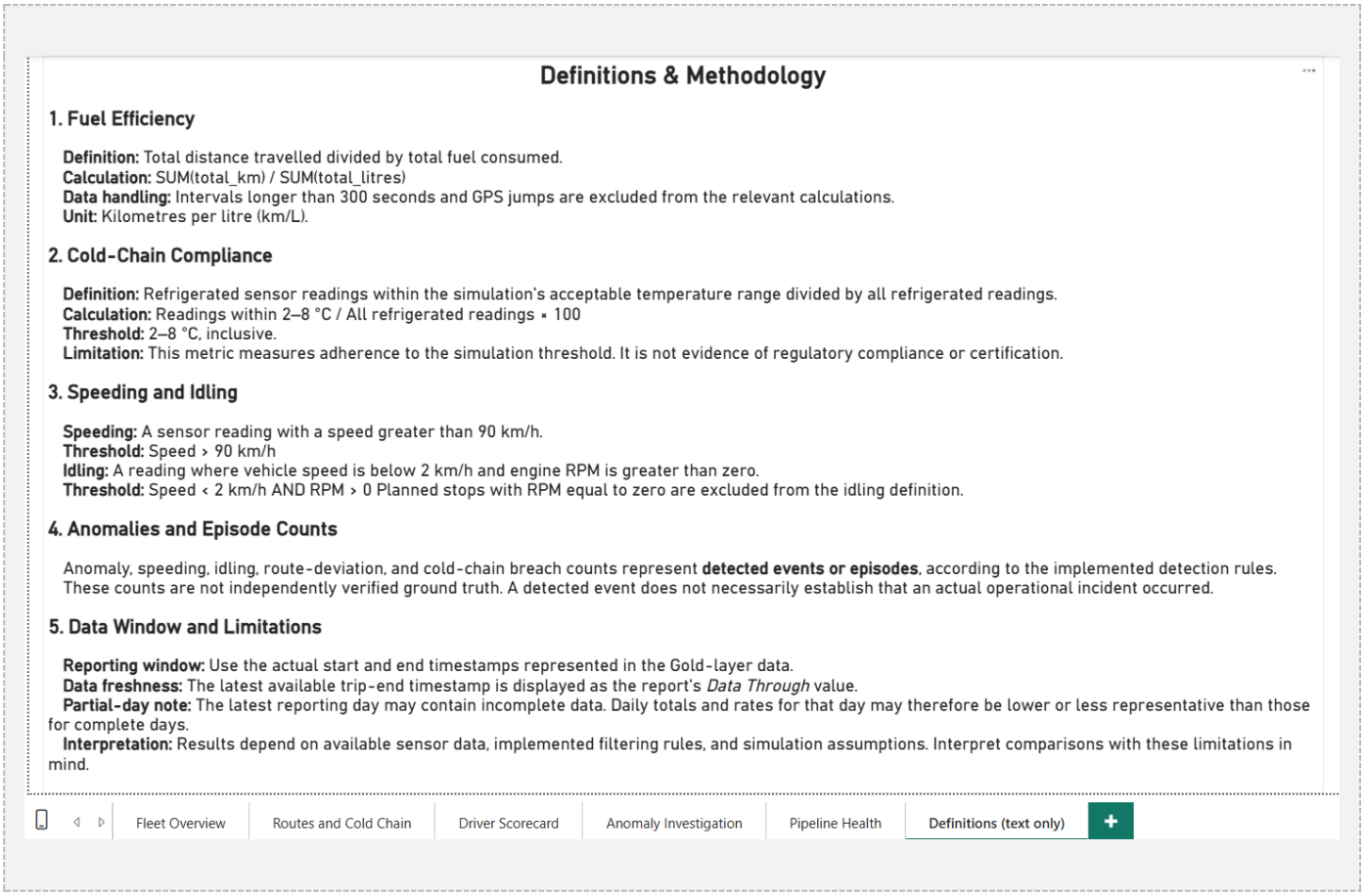
11.5 Page 5 — Pipeline Health

Purpose: let a data engineer or reviewer prove the numbers are trustworthy — that quality checks ran, that Gold agrees with Silver, that driver history loaded correctly, and that jobs finished in reasonable time. Cards show 28 PASS / 1 WARN / 0 FAIL quality checks, 100.00% reconciliation match, and 416 drivers with SCD Type 2 history (matching the expected count). A matrix breaks quality results down by check and table; a bar chart shows execution duration by job; and a details table shows the full Silver-to-Gold reconciliation (0 mismatches across 52,917 groups compared).



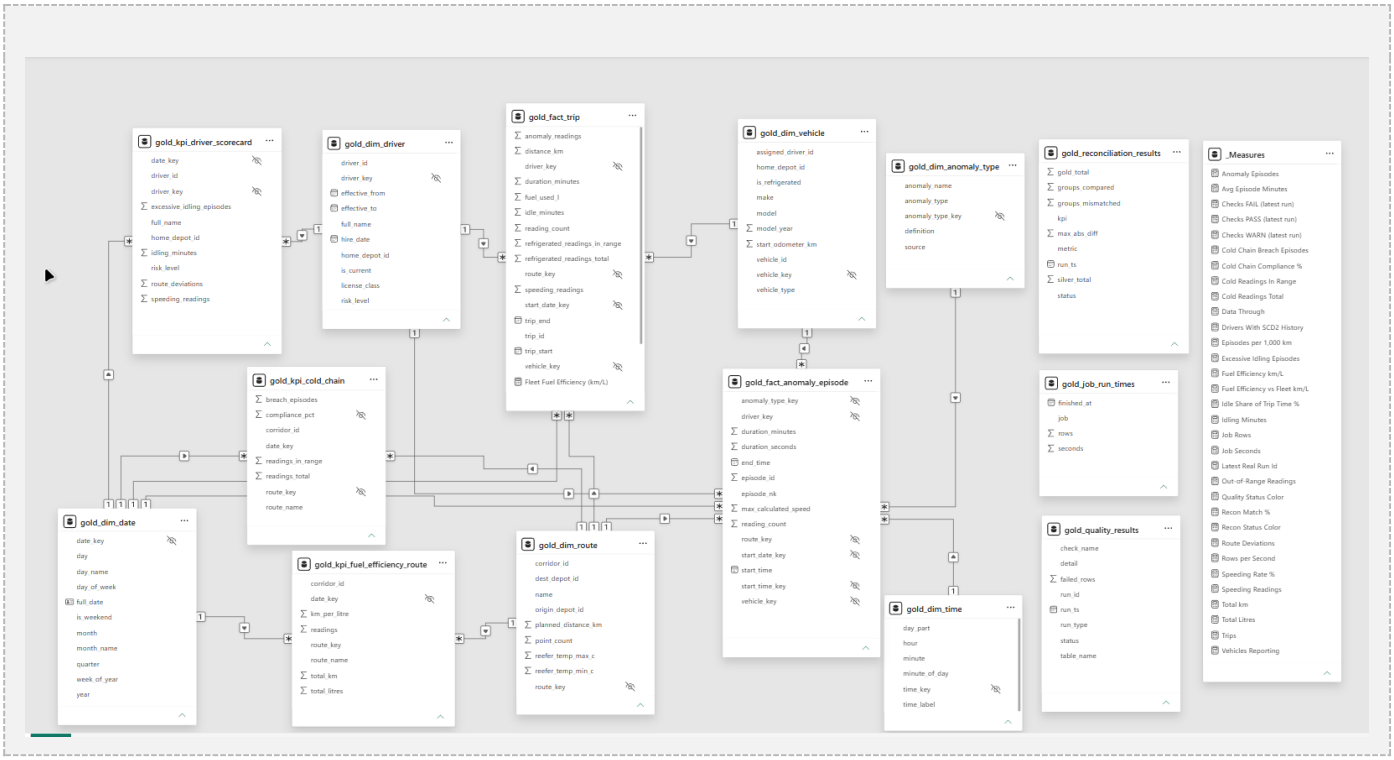
11.6 Page 6 — Definitions & Methodology

A text-only page, with no visuals, stating the rules behind every number on the other five pages — fuel efficiency, cold-chain compliance, speeding/idling thresholds, how anomaly counts should be read, and the data window — so that no reader of the dashboard has to guess what a metric means.



11.7 The Power BI model

Gold tables are imported into a single Power BI semantic model with defined relationships and DAX measures. Dimensions describe things (route, driver, vehicle, date, time, anomaly type); facts record events (trips, anomaly episodes); KPI tables hold pre-calculated daily numbers; and the three operational tables are deliberately left unrelated to the rest of the model, since they describe pipeline runs, not fleet activity.



11.8 Reported snapshot figures

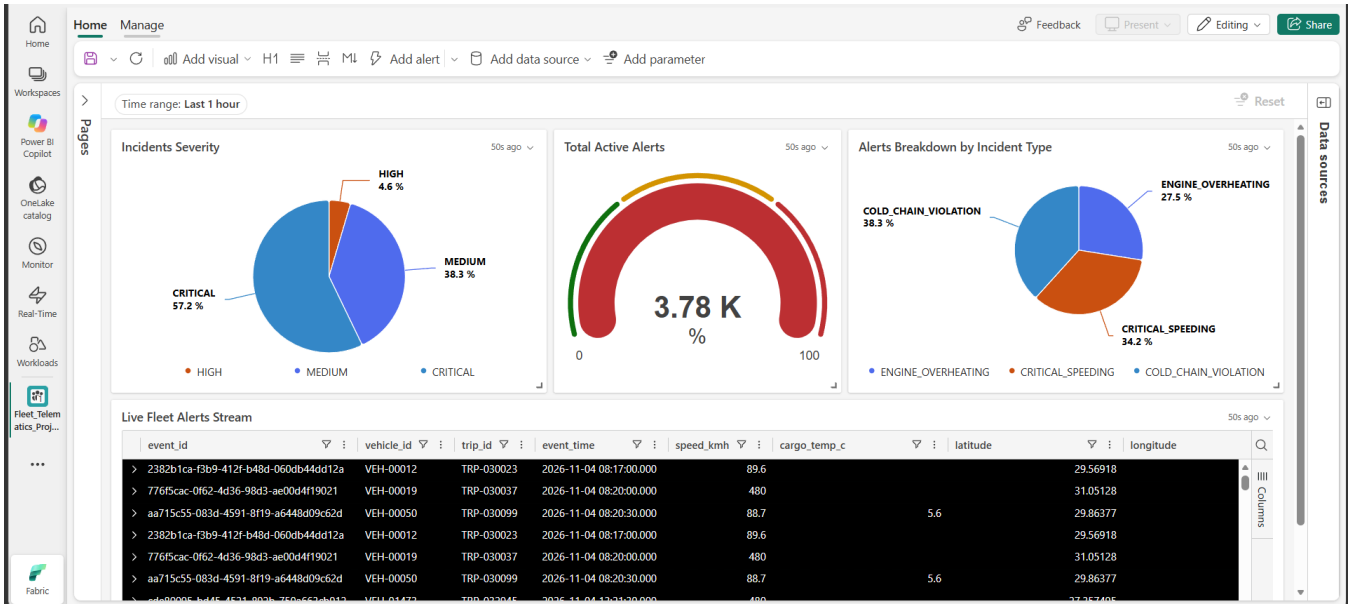
The figures below describe the dashboard's documented report state at the time of writing. They are a snapshot of that run, not a guaranteed result for every rerun — re-check them against the live tables before presenting.

Measure	Reported value
Fleet fuel efficiency	2.77 km/L
Cold-chain compliance	99.86%
Cold-chain breach episodes	163
Detected anomaly episodes	9,724
Vehicles reporting	5,000 (target 5,000)
Quality checks	28 PASS, 1 WARN, 0 FAIL
Reconciliation	100.00% match across 52,917 groups compared
Drivers with SCD Type 2 history	416 (expected 416)

Real-Time Fleet Alerts Dashboard

A real-time dashboard was developed directly in Microsoft Fabric to monitor incoming fleet alerts and provide immediate operational visibility. It displays incident severity distribution, total active alerts, and alert breakdowns by incident type, including engine overheating, critical speeding, and cold-chain violations. A live alerts table provides event-level details such as vehicle ID, trip ID, event time, speed, cargo temperature, and GPS coordinates.

The dashboard supports continuous monitoring of fleet conditions, helping operators identify critical incidents and investigate potential safety, vehicle, and refrigerated-cargo issues as events arrive.



[Microsoft Fabric Real-Time Fleet Alerts Dashboard]

12. Validation and Measured Evidence

The handbook's Testing & Validation pillar asks for evidence of data quality, failure handling, analytical correctness and pipeline behaviour — not just a working demo. This section summarizes the measured runs the team used to prove the pipeline behaves correctly under normal operation, incremental re-runs, mid-run failure, and backfill.

12.1 Batch ingestion: the five required runs

The handbook's testing guide asks for a minimum of five batch scenarios, including a failure/retry case and a backfill case. All five were executed against the v2 source system:

#	Scenario	Rows read	Inserted	Skipped
1	Initial load / backfill (whole day 2026-11-02)	4,014,197	4,014,197	0
2	Incremental #1 (new hour 12:00)	293,897	293,897	0
3	Incremental #2 (new hour 13:00, plus hour 12:00 re-extracted — idempotency check)	573,047	279,150	293,897
4	Failure + retry (new hours 14:00, 15:00; extractor killed mid-upload of hour 15:00; restart redid only the unfinished hour)	502,818	502,818	0
5	Backfill of an already-loaded hour (2026-11-02 10:00)	424,918	0	424,918

Total inserted from history extracts: 5,090,062 rows (= 5,090,062 distinct db_id values), against the handbook's Fleet-domain guide range of 4.35–17.4M. 69,971 rows were rejected and kept, with reasons matching exactly what the generator injected. A genuine, unplanned failure also occurred during Run 1: an hour-07 upload (36 MB) timed out; after switching uploads to 4 MB chunks, the next run correctly redid exactly the 17 hours left unfinished — both attempts are visible in extract_log.

12.2 Streaming: the golden run

A single, sustained streaming run — all 5,000 vehicles, simulated day 2026-11-04 (06:00–15:05), rate-limited to 700 events/second, run for 75.2 minutes — was used to validate the live path at scale, including a deliberate mid-run restart of the bridge.

Guide requirement (Fleet domain)	Recommended level	Measured result
Run duration (wall-clock)	60–90 min	75.2 min
Generation rate	500–1,000 events/s	700.0 events/s
Volume	2–5M events	3,157,802 events
Time windows crossed	4–5 (30–60s windows)	≈ 75 one-minute windows
Optimal extras	restart + late events	Bridge restarted mid-run; 9,353 late events injected

Measured in the cloud	Value

Guide requirement (Fleet domain)	Recommended level	Measured result
Rows in Bronze / distinct db_id for the run	3,157,802 / 3,157,802 — nothing lost, nothing duplicated across the restart	
Database-to-cloud latency	average 1.59 s; p50 1.58 s; p95 2.36 s; max 7.34 s	
Kafka lag during the run	median 690 messages; max 1,427 (about 1–2 seconds of data)	
Rejected at the API	43,340 (9,324 malformed, 12,372 missing, 12,448 null, 9,196 wrong type)	
Anomaly episodes injected	171 cold-chain, 391 idling, 281 route deviation	

The restart caused no gap and no duplicates in the delivered data, because the bridge commits Kafka offsets only after the cloud acknowledges receipt, so it resumed exactly where it left off. The bridge's own 10-second interval log is missing roughly 5,000 deliveries around the restart itself — a logging gap, not a data-loss gap, since Kafka and the cloud both show every event; the team has since fixed this by having the bridge write a final summary row on shutdown.

12.3 Cross-hop reconciliation

Row counts were compared at every hop from the generator to Gold, for both a small smoke test and the full batch and streaming runs:

Hop	Smoke test	Batch total	Golden run
Generator sent	12,325	5,090,062	3,157,802
PostgreSQL (via API)	12,325	5,090,062	3,157,802
Kafka topic (via CDC)	12,325	—	3,157,802
Bridge committed (cloud-acknowledged)	12,325	—	3,157,802
Extract files loaded	—	5,090,062	—
Bronze v2, distinct db_id	12,325	5,090,062	3,157,802
Rejected with a reason (= injected)	164	69,971	43,340

In total, bronze_telemetry_v2 holds 8,260,189 distinct source rows (3,170,127 from the streaming path plus 5,090,062 from the batch path). The dashboard documentation separately reports 8,146,714 valid Silver readings plus 113,475 rejected rows, which sum to the same 8,260,189 total — the two figures reconcile arithmetically, but should still be checked against the live tables before a new demonstration, since they come from different documents written at different times.

12.4 Gold quality, reconciliation and job timing

The documented Power BI snapshot shows 28 PASS, 1 WARN and 0 FAIL quality checks; 100.00% reconciliation across 52,917 groups compared; and 416 driver histories correctly represented with SCD Type 2. The one warning (range_sensor_values on telemetry, 32,638 rows) is by design — the pipeline deliberately keeps out-of-range sensor values visible instead of silently deleting them, so it is expected to appear.

Job	Duration (seconds)
NB02 TOTAL	263
NB02 fact_telemetry	114
NB04 reconciliation	103
NB03 quality_checks	83
NB04 build_kpis	31
NB02 fact_trip	24
NB02 fact_anomaly_episode	16

These are measurements from one specific run on shared Fabric trial capacity, not general performance guarantees — the trial capacity runs only a few Spark sessions at once, and an open interactive session was observed to block a loader run once (TooManyRequestsForCapacity).

13. Business Value and Conclusion

This project set out to answer a concrete business problem: fleet managers cannot see fuel efficiency, cold-chain safety and driver behaviour clearly when those signals arrive from disconnected systems. The pipeline built here takes simulated telemetry from 5,000 vehicles through a realistic source system, a cloud lakehouse, and a validated dimensional model, and lands it on a six-page Power BI dashboard that answers exactly the questions the business challenge posed:

- Fuel efficiency and route comparison, calculated correctly as a true fleet-wide ratio rather than an average of per-route percentages (2.77 km/L fleet-wide).
- Cold-chain compliance, scoped correctly to refrigerated trucks only, with the evidence behind each percentage visible (99.86% compliance, 163 breach episodes, backed by 2,415,864 refrigerated readings).
- Driver behaviour — idling, speeding and route deviations — broken down per driver and per depot, with the correct handling of drivers whose attributes changed mid-simulation.
- Anomaly investigation, letting an analyst move from "how many events happened" to "where, when and how long," down to individual episodes.
- Pipeline trust, on its own dedicated page, so a reviewer or engineer does not have to take the business numbers on faith: quality checks, Silver-to-Gold reconciliation, and job timings are presented alongside the business KPIs.

Equally important to the business value is what this document does not claim. The data is simulated, cold-chain compliance measures adherence to a simulation threshold rather than regulatory certification, and anomaly counts represent what the implemented detection rules found rather than independently verified real-world incidents. Several open items — most importantly the two differing anomaly-evaluation results and a handful of unconfirmed dashboard measures — are documented in Section 13 precisely so they are resolved before this platform is presented as a finished, production-ready system rather than a graduation-project demonstration of the underlying engineering.

Taken together, the project demonstrates every required pillar of the SIC 8 Big Data Engineering handbook end to end: a reproducible domain-specific generator with controlled anomalies; dual-path ingestion (streaming and scheduled batch) with dead-letter handling, logging and retry backoff; distributed PySpark processing with explicit schemas, deduplication and enrichment; a Delta Lake star schema with Type 1 and Type 2 dimensions, quality gates and reconciliation; and a business-facing Power BI dashboard — with measured evidence gathered at every hop from the generator to the final visual.

14. Team Responsibilities

The table below records each team member's technical area of ownership, matching the four areas of implemented work described throughout this document. Names and final, verified contribution details should be completed by the team before submission — they are intentionally left blank here rather than attributed by assumption.

Team member	Role / area	Implemented contribution	Evidence / files
Shahesta Mohamed	Source generation and ingestion (Person 1)	Fleet generator, FastAPI ingestion API, PostgreSQL source database, Debezium CDC + Kafka, Fabric bridge, hourly extractor, Fabric notebook launcher	person1_generation_and_ingestion.md; person1_v2_source_system.md; evidence/person1/
Ahmed Mohamed	Silver processing and anomaly detection (Person 2)	Schema parsing, validation, rejected-record handling, deduplication, incremental streaming write, enrichment, movement analytics, anomaly rules, episode aggregation, evaluation	person2.md; evidence/person2/
Yahya Moustafa	Gold model, KPIs and data quality (Person 3)	Star schema (dimensions + facts), SCD Type 2 driver history, three Gold KPI tables, data quality gate, Silver-to-Gold reconciliation, job-timing log	person3_gold_layer.md
Abdullah Fathy	Power BI dashboard, orchestration and integration (Person 4)	Six-page Power BI report (Fleet Overview, Routes & Cold Chain, Driver Scorecard, Anomaly Investigation, Pipeline Health, Definitions & Methodology), semantic model, DAX measures	Fleet Analytics Dashboard Documentation; docs/images/